

If you know the ID of a particular object, you can use the method `Topic.objects.get()` to retrieve that object and examine any attribute the object has. Let's look at the text and `date_added` values for Chess:

```
>>> t = Topic.objects.get(id=1)
>>> t.text
'Chess'
>>> t.date_added
datetime.datetime(2022, 5, 20, 3, 33, 36, 928759,
                  tzinfo=datetime.timezone.utc)
```

We can also look at the entries related to a certain topic. Earlier, we defined the topic attribute for the Entry model. This was a `ForeignKey`, a connection between each entry and a topic. Django can use this connection to get every entry related to a certain topic, like this:

```
❶ >>> t.entry_set.all()
<QuerySet [<Entry: The opening is the first part of the game, roughly...>,
<Entry:
In the opening phase of the game, it's important t...>]>
```

To get data through a foreign key relationship, you use the lowercase name of the related model followed by an underscore and the word `set` ❶. For example, say you have the models `Pizza` and `Topping`, and `Topping` is related to `Pizza` through a foreign key. If your object is called `my_pizza`, representing a single pizza, you can get all of the pizza's toppings using the code `my_pizza.topping_set.all()`.

We'll use this syntax when we begin to code the pages users can request. The shell is really useful for making sure your code retrieves the data you want it to. If your code works as you expect it to in the shell, it should also work properly in the files within your project. If your code generates errors or doesn't retrieve the data you expect it to, it's much easier to troubleshoot your code in the simple shell environment than within the files that generate web pages. We won't refer to the shell much, but you should continue using it to practice working with Django's syntax for accessing the data stored in the project.

Each time you modify your models, you'll need to restart the shell to see the effects of those changes. To exit a shell session, press `CTRL-D`; on Windows, press `CTRL-Z` and then press `ENTER`.

TRY IT YOURSELF

18-2. Short Entries: The `__str__()` method in the Entry model currently appends an ellipsis to every instance of Entry when Django shows it in the admin site or the shell. Add an if statement to the `__str__()` method that adds an ellipsis

(continued)